

PRODUCT & RELEASE REVIEW

Codexa Bounded Causal Change Intelligence

Bounded, deterministic relationship evidence for Codexa's existing change-governance flow.

PR summary for codex/context-intelligence-next against main.

- Base: c306c1d (v0.18.0)
- Validated feature source head: 0474276; published as 869257e
- CI hardening head: 41118bb; published as 9c29408
- First review hardening head: a6ac9d0; published as 05a1dc3
- Final code head: balf090
- Final code tree: fdf2f4f569f11f93f12a59f15af461b46d5e6904
- Branch delta before this evidence refresh: 62 files, 5,510 insertions, 267 deletions
- Commits before this evidence refresh: 8 Conventional Commits - 6591e60 - feat: add bounded causal change intelligence - 0474276 - test: accept SDK validation wording - d18637f - docs: add context intelligence PR summary - 41118bb - fix(bootstrap): preserve first terminal condition - 3e2608a - docs: refresh context intelligence review evidence - a6ac9d0 - fix(workflow): preserve capped membership authority - 3fa989c - docs: record workflow membership review fix - balf090 - fix(workflow): retain exact capped membership

Executive Outcome

Codexa now explains a proposed or completed change through bounded, source-backed causal evidence: the declared edit target, the runtime or workflow path it participates in, nearby blast-radius evidence, and the tests that can verify it. The same evidence model follows change planning, post-edit review, committed change review, and proof without granting itself edit authority or changing a readiness verdict.

This keeps Codexa in its lane. GitNexus- and Graphify-style relationship value is delivered inside Codexa's existing local governance and completion flow; there is no graph database, graph query language, vector service, UI, new MCP tool, cross-repository crawler, or embedded model call.

The implementation uses a functional-core/imperative-shell design: immutable facts, explicit typed transitions, deterministic fingerprints, pure bounded selection, stable tie-breaking, and side effects confined to existing index and snapshot boundaries.

Highest-ROI Adjustments

1. One evidence model across the completion lifecycle

- Adds schema-validated evidence bundles to change plans, task snapshots, post-edit review, committed review, and proof cards.
- Preserves original edge direction and provenance on every chain segment.
- Separates caller-authorized edit targets from inferred read dependencies and verification targets. Evidence can explain scope; it cannot widen scope.
- Keeps evidence advisory: it cannot promote authority, readiness, or verdicts.
- Carries stable snapshot, task, chain, and bundle identities through MCP compaction, with explicit counts, gaps, and truncation receipts.

2. Framework execution surfaces become useful graph evidence

- Indexes Commander CLI roots, aliases, subcommands, singleton and namespace forms, plus CommonJS destructuring and namespace usage.
- Indexes MCP tool registrations, including bounded helper-wrapper discovery.

- Resolves lexical declaration identity so shadowed `program`, `Command`, `server`, and helper names cannot create false workflows.
- Rejects control-character names and unrelated look-alike receivers.
- Uses one-pass workflow evidence indexes and production-first bounding so test fan-out cannot evict the runtime implementation from a workflow packet.

3. Deterministic, database-free causal traversal

- Bounds a bundle to 3 chains, 6 segments per chain, depth 4, 12 paths, 4 tests, 4 gaps, 4,096 visited nodes, 16,384 examined edges, and 8 analyzed targets.
- Preserves deterministic output under edge-order permutations and diamond graphs through canonical ordering and explicit tie-breaking.
- Stores canonical adjacency references in `Uint32Array` form and materializes neighbor objects only during the bounded traversal.
- In a 500,000-edge fixture, retained adjacency fell from about 160 MiB to exactly 4,000,000 bytes (about 3.8 MiB), roughly a 40x reduction; setup stayed near 1.3 seconds. Retained references are capped at 8 bytes per edge.

4. Safe index and transport evolution

- Advances the derived index revision to 3. Revision-2 indexes remain readable but stale and rebuild; revision-3 indexes fail closed if exact workflow membership is missing or malformed.
- Persists only the paths omitted by public workflow caps in an internal `workflowMembershipSpill` map. Every public workflow, MCP, facts, and relational projection remains unchanged and bounded.
- Uses the same 512 MiB UTF-8 limit in the writer and loader. Oversized indexes fail before atomic replacement, so the writer cannot publish an artifact the loader would reject.
- Preserves workflow truncation receipts and evidence identity through compact MCP profiles.
- Keeps existing public commands and MCP tools intact; the change enriches their evidence rather than introducing another overlapping surface.
- Updates `@modelcontextprotocol/sdk` from 1.29.0 to 1.30.0. Codexa's response budget remains at most 512 KiB, well below the SDK's 10 MiB stdio input cap. The resolved dependency audit is clean.

Human Workflow Simulation

A disposable JavaScript checkout service was indexed and changed as a developer would use it. Codexa identified a Commander `checkout` workflow and an MCP `calculate_checkout` workflow across 6 files, 9 symbols, and 51 usage sites.

The simulated change moved a loyalty discount before tax. The first attempt edited an undeclared test and correctly received a blocking scope-drift review. After reverting and replanning with both source and test explicitly declared, the implementation and 3 Node tests passed, post-edit review returned `continue`, the proof card was ready with zero gaps, and strict committed review of HEAD~1. HEAD returned PASS. The repository was re-indexed and the workflow/review sequence repeated after the initial feature hardening.

The current tool environment does not expose a live external Graphify MCP server, so no external Graphify call is claimed. Codexa's real stdio and coexistence suites exercised primary-server behavior, initialization isolation, transport, and preservation of unrelated MCP configuration: 30 checks passed with 1 intentional skip.

Verification on the Exact Source Head

Frozen validation on final source head `ba1f090`:

- Build, typecheck, lint, release-path, publish, privacy, and `git diff --check` passed.
- Final monolithic Vitest rerun: 110/110 files, 1,241 passed, and 1 intentional skip (1,242 total). The earlier frozen sharded run had 1,240 pass, 1 skip, and one `command-process-tree.test.ts` temporary-file `ENOENT`; its immediate isolated rerun passed 3/3. The passing monolithic run used a writable task-specific npm cache required by this sandbox. Exact-head GitHub CI remains the merge authority.
- Claude integration: 28 command smokes and 89 hook smokes (117 total).
- Startup context gate: project kernel 2,823/3,072 bytes; 3/23 direct tools; measured reductions of 87.2% and 69.9% on the guarded surfaces.

- `npm audit`: 0 vulnerabilities.
- Clean public snapshot, package/plugin hygiene, and one-commit source checks passed.
- The focused final membership, schema, extraction, and artifact-size suites passed: 9 files and 49 tests before the size guard, 4 files and 14 tests after it, plus the final revision-2 compatibility schema run at 9/9.

Additional release evidence:

- Eval gate: 21 scenarios, score 1, and `rawRgBetter=0`.
- CI-scaled hot-path benchmark: all 12 gates passed. One unscaled watch item, `cli.session_start` p95, measured 1,012 ms against a 1,000 ms base target and remained below the 1,500 ms CI gate.
- Pinned v0.12 transport comparison: all gates passed; tool-list reduction 85.3%, startup advertisement/discovery reduction 55.9%, first-result reduction 75.0%, and repeated-result reduction 86.6%.
- Focused scale regression: 22/22 passed; execution-surface, MCP transport, and coexistence regressions passed.

A real final-head index of Codexa contained 481 files, 5,477 symbols, 98,369 usage sites, and 41 workflows in 67,292,652 bytes. Only 2 workflows needed spill membership, for 9 spill entries (5 unique paths) and a maximum of 5 on one workflow. `workflow-path` recovered `command index` and `command semantic-index` from `tests/task-lifecycle.test.ts` even though that test was outside the displayed 20-test cap. `change-plan` then emitted both workflows as required checks while keeping only the caller-declared test editable.

The SDK update initially exposed one brittle assertion that accepted only the old validator wording. `0474276` widened that test to accept both stable wording forms, after which the complete security gate passed without a waiver.

The first PR run then exposed a pre-existing Windows bootstrap race: an output cap could start process-tree termination, but the still-armed stage timer could fire during slow `taskkill` teardown and relabel the earlier output-cap failure as a timeout. `41118bb` preserves the first terminal condition and makes the regression deterministic by forcing teardown across the timer boundary. The focused test passed 3/3 with typecheck, lint, release-path verification, and `git diff --check`; independent review found no P0-P2 issue. No deadline or log bound was relaxed.

Automated PR review then found that the 40-file public `relatedFiles` cap could hide a later production member from internal `change-plan`, `post-edit`, `proof`, and `required-check` matching. `a6ac9d0` keeps the public/MCP caps unchanged while matching against retained workflow steps and prioritizing every scoped member up to the public 64-file plan bound. Regressions cover production target #41 and a 21-member scope where only the former receipt omission is edited. Five focused files and 36 tests passed with typecheck, lint, and diff checks; independent re-review found no P0-P2 issue.

A fresh exact-head review then demonstrated a stricter case: tests beyond the 20-test cap could still disappear from internal matching because typed workflow steps and production-first related files were also bounded. `ba1f090` replaces that inference with exact pre-cap membership spill data, threads it through plan, workflow query, evidence, `post-edit`, `proof`, and `required-check` consumers, and adds regressions for test #21, test-symbol-only scope, and dependent tests seeded by typed `TEST_COVERS_WORKFLOW` steps. Legacy revision-2 artifacts rebuild, and revision-3 artifacts cannot load with incomplete spill data. Independent semantic review reported no P0-P3 finding.

Risk-Budgeted Review

Finding weights are critical 8, high 5, medium 3, and low 1. Merge requires no critical or high finding and no more than 4 residual points.

Independent review found and closed three release-significant risks before the source head was published: lexical receiver shadowing in Commander/MCP extraction, test-heavy workflow truncation evicting production files, and eager object-heavy adjacency retention. Regression tests cover each fix. The final correctness, authority, scale, and dependency reviews report 0 critical, 0 high, and 0 medium findings. The first capped-membership P2 and the Windows failure-cause race were closed. The second capped-membership P2 is code-addressed with focused regressions and clean independent re-review; its GitHub thread remains pending final-head publication and exact-head CI/review.

Residual score: **2/10 (within budget)**. Low-risk watch items are the one-time `O(E log E)` canonical typed-array sort used to build a deterministic adjacency index and pathological repetition of large hidden workflow memberships. The latter is byte-bounded and rejected before publication; traversal and retained adjacency storage remain explicitly bounded.

Scope Honesty

- New language or framework support: Commander and MCP execution-surface recognition inside existing JavaScript/TypeScript indexing; no new language.
 - Embedded LLM call: no.
 - Public surface: existing CLI/MCP responses gain bounded evidence; no new MCP tool or configuration knob.
 - Dependency change: no new direct dependency; existing MCP SDK updated to 1.30.0 with audited transitive resolutions.
 - Linked issue: none; this is an owner-directed enhancement phase.
-

Rollout, Deployment, and Live Verification

- 1. Require all six exact-head GitHub jobs: `check`, `package-smoke`, `benchmark`, and `worktree-bootstrap` on Ubuntu, macOS, and Windows.
- 2. Resolve every actionable review thread and keep the final residual score at or below 4 before merge.
- 3. Merge the feature PR through the protected `main` branch.
- 4. Use only `.github/workflows/release-please.yml` on `main` with `secrets.RELEASE_PLEASE_TOKEN` to create or update the expected `chore(main): release 0.19.0` PR.
- 5. Inspect and merge that checked release PR. Let the resulting GitHub Release trigger `.github/workflows/npm-publish.yml` with `secrets.NPM_TOKEN`.
- 6. Verify the GitHub release/tag, npm registry version, MCP Registry entry, fresh install, CLI version/startup, and a real MCP initialization/tool call.

No manual tag or direct npm publish is part of this rollout.

Rollback

Before publication, revert the feature commits and rebuild derived indexes under the restored code version. After publication, preserve npm immutability: revert on `main`, ship a corrective patch through the same secret-backed flow, and deprecate the affected version only for a material safety risk.